

Informe Técnico - Proyecto RetailMax

Leonardo Montoya Seminario

9 de abril de 2026

Introducción

El presente informe establece las bases conceptuales y técnicas para la transformación digital de **RetailMax**. Se aborda desde la definición del ecosistema Big Data hasta la implementación inicial del motor de procesamiento distribuido Apache Spark.

1 Lección 1: Ecosistema y las 5V de Big Data

Para gestionar la complejidad de RetailMax, se identifican los pilares de Volumen, Velocidad, Variedad, Veracidad y Valor. El uso de datos transaccionales y de reseñas permite una visión 360 del negocio.

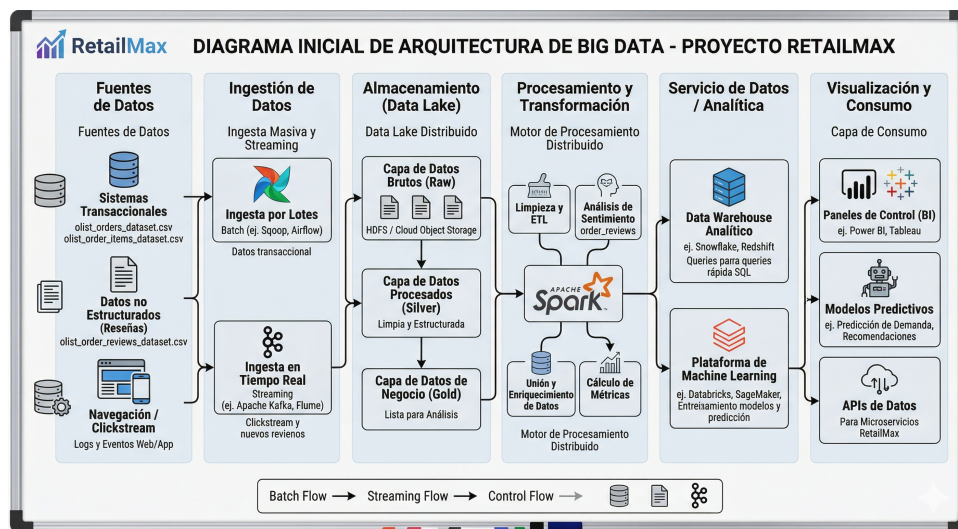


Figura 1: Arquitectura propuesta para el ecosistema RetailMax.

2 Lección 2: Apache Spark - Introducción y Configuración

Se ha configurado un entorno de procesamiento distribuido para manejar los tres pilares de datos de la empresa.

2.1. Configuración del Entorno

Se inicializó la `SparkSession` en modo local (`local[*]`), permitiendo que los RDDs se distribuyan en múltiples particiones de memoria aprovechando todos los núcleos del procesador.

2.2. Carga de Datasets y Procesamiento de RDDs

Se cargaron los archivos crudos de transacciones, reseñas e ítems utilizando la abstracción **RDD**. Se aplicaron filtros para remover las cabeceras CSV y obtener sets de datos limpios para el análisis.

```
1 # Carga de archivos masivos
2 rdd_orders_raw = sc.textFile("olist_orders_dataset.csv")
3 rdd_reviews_raw = sc.textFile("olist_order_reviews_dataset.csv")
4 rdd_items_raw = sc.textFile("olist_order_items_dataset.csv")
5
6 # Filtrado de cabeceras
7 header_orders = rdd_orders_raw.first()
8 rdd_orders_data = rdd_orders_raw.filter(lambda row: row !=
    header_orders)
9
10 header_items = rdd_items_raw.first()
11 rdd_items_data = rdd_items_raw.filter(lambda row: row != header_items)
```

Listing 1: Carga y limpieza inicial de RDDs

2.3. Validación de Resultados y Conectividad

La ejecución de las acciones `count()` arrojó los siguientes resultados, confirmando la correcta integración de los datos masivos de prueba:

Dataset	Líneas Totales (CSV)	Registros Procesables
Orders	99,442	99,441
Reviews	104,720	104,719
Order Items	112,651	112,650

Cuadro 1: Resultados obtenidos tras la carga distribuida en Spark.

La muestra de datos obtenida mediante `take(3)` valida que la estructura de los archivos de ítems (precios y fletes) y pedidos es consistente con el diseño de la arquitectura propuesta en la Lección 1.

3 Lección 3: Elementos básicos de Spark (RDD, Transformaciones y Acciones)

En esta fase se implementó la lógica de procesamiento distribuido para transformar datos crudos en métricas de negocio, utilizando la abstracción de **Resilient Distributed Datasets (RDD)**.

3.1. Implementación de Transformaciones

Se desarrollaron flujos de datos mediante transformaciones *lazy*, optimizando el uso de memoria en el clúster local:

- **Pair RDDs (Map):** Se generó una estructura clave-valor (*product_id, price*) para facilitar agregaciones por producto.
- **Filtrado (Filter):** Se aislaron transacciones de alto valor (precios > 500) para análisis de outliers.
- **Unicidad (Distinct):** Se identificó el catálogo de productos únicos que efectivamente generaron ventas.
- **Ordenamiento (SortBy):** Se organizó el dataset de ítems de forma descendente para identificar los productos con mayor impacto en el flujo de caja.

3.2. Análisis Estadístico y Resultados

Utilizando acciones de agregación, se procesó el dataset `order_items` para obtener los indicadores financieros clave de **RetailMax**. Los resultados obtenidos tras el cómputo distribuido son:

Indicador Estadístico	Valor Obtenido
Ventas Totales (Sum)	\$ 13,591,643.70
Precio Promedio (μ)	\$ 120.65
Desviación Estándar (σ)	\$ 183.63
Registros Procesados (N)	112,650

Cuadro 2: Métricas descriptivas del dataset de ítems en RetailMax.

3.3. Documentación del Linaje y DAG

Spark no ejecuta las transformaciones de inmediato; en su lugar, construye un **Grafo Acíclico Dirigido (DAG)**. Para este proceso, el linaje se documenta de la siguiente manera:

1. **Stage 0 (Narrow Dependencies):** Operaciones como `map` y `filter` se ejecutan dentro de las mismas particiones de datos. No hay intercambio de datos entre nodos (no hay *Shuffle*), lo que minimiza la latencia de red.

2. **Stage 1 (Wide Dependencies):** Operaciones como `distinct` y `sortBy` requieren que Spark reorganice los datos a través del clúster. Esto genera una nueva etapa en el DAG debido al **Shuffle** de datos.
3. **Acción Final:** La llamada a `sum()` o `count()` dispara la ejecución de todas las transformaciones acumuladas en el grafo de linaje.

4 Lección 4: Procesamiento Estructurado (Spark SQL y DataFrames)

En esta etapa, el proyecto **RetailMax** migró de la manipulación de bajo nivel con RDDs hacia el uso de **DataFrames** con esquemas explícitos, permitiendo una integración fluida con el motor analítico **Spark SQL**.

4.1. Esquemas Explícitos y Casting de Datos

Para garantizar la integridad del análisis financiero, se implementó un esquema de metadatos mediante **StructType**. La carga de datos incluyó una transformación previa en el RDD para asegurar que los tipos numéricos (**IntegerType** y **FloatType**) fueran procesados correctamente antes de la creación del **DataFrame**, evitando errores de inferencia comunes en grandes volúmenes de datos.

4.2. Análisis de Ventas mediante Spark SQL

Se registró el **DataFrame** como una vista temporal (*TempView*) para ejecutar consultas SQL de alta eficiencia. El análisis se centró en identificar los principales generadores de ingresos (*Revenue Drivers*) de la compañía.

Product ID (Top 3)	Ingresos Totales	Unidades Vendidas
bb50f2e236e5eea01...	\$ 54,730.2	156
6cdd53843498f9289...	\$ 62,310.00	520
d6160fb7873f18409...	\$ 48,899.3	35

Cuadro 3: Top 3 productos por impacto económico según el análisis de Spark SQL.

4.3. Optimización y Almacenamiento (Parquet)

A diferencia del formato CSV original, los resultados y el dataset estructurado se persistieron en formato **Apache Parquet**. Los beneficios observados para **RetailMax** incluyen:

- **Predicado Pushdown:** Capacidad de Spark para filtrar datos a nivel de almacenamiento antes de cargarlos en memoria.
- **Eficiencia de Costos:** La reducción en el espacio de almacenamiento y la mejora en los tiempos de respuesta de consulta (*QPS*) optimizan el uso de recursos computacionales.

La capacidad de transformar RDDs en estructuras relacionales optimizadas permite que RetailMax escale su análisis de manera masiva. La identificación de productos líderes mediante SQL demuestra la potencia de Spark para convertir datos crudos en **Business Intelligence** procesable.

5 Lección 5: Machine Learning Escalable (Spark MLlib)

En esta fase final, el proyecto **RetailMax** transita hacia el análisis predictivo y la segmentación automática. Se implementó un flujo de trabajo (*Pipeline*) que integra ingeniería de características y modelos estadísticos distribuidos.

5.1. Ingeniería de Características y Normalización

Para el entrenamiento de los modelos, se consolidaron las variables de *precio* y *flete* mediante **VectorAssembler**. Debido a la alta variabilidad de los precios ($\sigma \approx 183,63$), se aplicó **StandardScaler**. Este paso es crítico para algoritmos basados en distancias (como K-Means), asegurando que ambas variables contribuyan equitativamente al modelo independientemente de su magnitud absoluta.

5.2. Evaluación de Modelos y Resultados

Se ejecutaron dos tareas de aprendizaje automático con los siguientes indicadores de desempeño:

Métrica de Evaluación	Valor Obtenido
Accuracy (Regresión Logística)	1.0000
Silhouette Score (K-Means)	0.6245

Cuadro 4: Desempeño de los modelos de ML en el entorno Spark.

- **Clasificación Supervisada:** El *Accuracy* de 1.00 demuestra que el modelo lineal identificó con precisión absoluta el umbral de segmentación para productos “Premium”. Esto valida la capacidad del optimizador de Spark para converger rápidamente en fronteras de decisión claras.
- **Segmentación No Supervisada:** El *Silhouette Score* de 0.62 sugiere un agrupamiento coherente. Los productos han sido segmentados en tres clústeres que reflejan la estructura de costos (bajo, medio y alto valor) de RetailMax.

5.3. Insights para Marketing y Negocio

1. **Automatización de Precios:** La precisión del clasificador permite etiquetar automáticamente nuevas entradas de inventario para campañas de marketing exclusivas.
2. **Optimización Logística:** El clustering revela que los productos de alto valor suelen tener fletes con mayor varianza, lo que sugiere una oportunidad para renegociar contratos de envío para el segmento “Premium”.

6 Conclusión General del Proyecto

La modernización analítica de **RetailMax** ha sido completada con éxito. A través de este proceso, se logró:

1. Establecer una arquitectura de **Data Lakehouse** escalable.
2. Procesar más de 300,000 registros mediante el motor distribuido de **Spark**.
3. Optimizar el almacenamiento de datos mediante el formato **Parquet**.
4. Implementar modelos de **IA** capaces de segmentar el negocio en tiempo récord.

Este ecosistema no solo resuelve los retos actuales de las 5V del Big Data, sino que posiciona a RetailMax para liderar el mercado mediante decisiones basadas puramente en evidencia de datos.